
🔐 Amazon CloudFront – Signed URLs & Signed Cookies

Amazon CloudFront offers **Signed URLs** and **Signed Cookies** to **secure access to private content** that is distributed through CloudFront. These mechanisms help restrict who can access your protected files and **for how long**.

🚧 Why Use Signed URLs or Signed Cookies?

When serving private content (e.g., videos, PDFs, software downloads), you don't want everyone to access it. You want to:

- Restrict access to **authenticated users only**
- Limit access **time**
- Prevent **unauthorized sharing of links**

📝 1. Signed URLs

✅ Use Case:

Use **Signed URLs** when:

- You want to **grant access to a single file** (e.g., /videos/course1.mp4)
- You are delivering downloadable files like PDFs or images

🛠️ How It Works:

- You create a **custom URL** for your content that includes:
    - **Policy or Canned Policy**
    - **Signature**
    - **Key-Pair ID**
    - **Expiration Timestamp**
- CloudFront verifies the request using your public key and allows access only if the request is valid and not expired.

🧪 Sample Signed URL Structure:

https://d111111abcdef8.cloudfront.net/resource.jpg
?Expires=1700000000
&Signature=abc123xyzSignature
&Key-Pair-Id=APKAExampleKey

🍪 2. Signed Cookies

✅ Use Case:

Use **Signed Cookies** when:

- You want to **grant access to multiple restricted files** (e.g., all files in a directory: /videos/*)
- You are streaming video or managing session-based content (e.g., HLS playback)

🛠️ How It Works:

- You set three HTTP cookies in the user's browser:
    - CloudFront-Policy
    - CloudFront-Signature
    - CloudFront-Key-Pair-Id
- Once set, any request from that browser to private CloudFront content will be authenticated automatically.

🔐 3. Steps to Implement Signed URLs / Cookies

✅ Step 1: Create a CloudFront Distribution

- Configure **Origin** (S3 or custom)
- Enable **Restrict Viewer Access** = Yes (use signed URLs or cookies)

✅ Step 2: Create Key Pair

- Create a **trusted signer** in IAM (CloudFront key pair)
- Use the private key in your app to **sign** the URL or cookie
- CloudFront uses the **public key** to verify authenticity

✅ Step 3: Generate Signed URL / Cookie

You can use:

- AWS SDK (Python, Node.js, etc.)

- AWS CLI
- Custom app logic using OpenSSL (for low-level control)

🎯 Key Differences: Signed URL vs. Signed Cookie

Feature	Signed URL	Signed Cookie
Use Case	One file	Multiple files
Delivery Type	File download or single view	Streaming or bulk access
Control Per User	Per link	Session-wide via cookies
Ease of Revocation	Easier	Slightly harder

📌 Security Features

- **Expiration**: Set expiry times in minutes or hours
- **IP Restriction**: Allow access only from certain IP addresses
- **Path Restriction**: Limit access to specific paths like /premium/*

💡 Example Scenarios

Scenario	Use
Paid video course	Signed Cookies for /videos/*

One-time software download	Signed URL
Granting access to sensitive PDF	Signed URL with short TTL
Secure video streaming	Signed Cookies with session control

🧪 Demo/Lab Ideas

- ✅ Serve a file from an S3 origin behind CloudFront with signed URL.
- ✅ Create a Lambda or Python function that generates a signed URL for a user.
- ✅ Apply signed cookies in a web application and restrict access to all files under /private/.

\